Assignment 1: Testing Software

by Prof. Peter Alvaro
edits by Ryan Leung and Dr. Kerry Veenstra
CSE 13S, Fall 2026
Document version 1 (changes in Section 7)

Due Wednesday, October 14th, 2026, at 11:30 pm

Contents

1 Introduction **1**

2 Copy the Resource Files **2**

3 Counting Birds With a `calc` Program **3**

4 The Spec **4**

5 Shell Scripting **4**
5.1 Which Commands to Script 4
5.2 The First Script 6
5.3 Running the First Script 7
5.4 Create a Second Script 7
5.5 How to Run All Scripts 7

6 Your Task **8**
6.1 Write Test Scripts 8
6.2 Scoring 8
6.3 questions.txt 9
6.3.1 Question 1 9
6.3.2 Question 2 9
6.4 Submit: add/commit/push/submit commit ID 9

7 Revisions **9**

1 Introduction

Brad Daylight, junior software engineer at Birdcat LLC, has recently inherited responsibility for maintaining a collection of programs written by people who no longer work for the company. It is now *his* job to make sure that this software works correctly, even though he had no role in the decisions that were made writing the code. (Read about "technical debt": `https://en.wikipedia.org/wiki/Technical_debt`.)

To make matters worse, this software was written in the C programming language, a memory-unsafe language from the late 1900s that Brad has only this quarter started to learn. In fact, in some cases, he does not even have the source code anymore, but needs to test binary programs without looking at their implementation!

> This assignment is different: you will not be writing any C code, but you will be writing *tests*.

2 Copy the Resource Files

Before continuing the story, let's get ready to run some examples.

☐ **To do:** Synchronize the **resources** repository in GitHub. Then copy the files for asgn1 into your repository. **Look at the `cp` command. Be certain to include the `-r` command-line option.** (It means "recursive," and will ensure that the subdirectories of the assignment will be copied.)

```
$ cd /wo tab /co tab ⏎
$ cd 13s/resources
$ git pull
$ cd ..
$ cp -r resources/asgn1/* yourcruzid/asgn1
$ cd yourcruzid/asgn1
$ ls
```

Did you forget the `-r` option in the `cp -r resources` command? Yes? Rerun the command with the `-r` option.

Below is the directory structure for the `asgn1` assignment. Eventually you will be writing test scripts in the `13s/yourcruzid/asgn1/tests` directory, but first you can run a couple examples in the appropriate `13s/yourcruzid/compiled_cats_???` directory.

Remember that you can use these commands to change where you are in the directory tree:

`$ pwd`	— Tells you where you are in the directory structure.
`$ cd` *directory*	— Change to a different directory (to the *right* in the diagram below).
`$ cd ..`	— Change back to a parent directory (*left and up* in the diagram).

```
13s
├── yourcruzid
│   └── asgn1
│       ├── compiled_cats_arm ..... Executable files for M1/M2/M3-based Apple laptops
│       │   ├── babycat
│       │   ├── blackcat
│       │   └── 18 more
│       ├── compiled_cats_hub ..... Executable files for GitHub
│       │   ├── babycat
│       │   ├── blackcat
│       │   └── 18 more ...
│       ├── compiled_cats_x86 ..... Executable files for Intel- and AMD-based laptops
│       │   ├── babycat
│       │   ├── blackcat
│       │   └── 18 more ...
│       ├── questions.txt
│       ├── runner.sh ..... Run test scripts with this script
│       └── tests
│           ├── test_3_4.sh ..... The first test script
│           ├── test_xyz_4.sh ..... The second test script—you make this in Section 5.4.
│           └── more ...
└── resources
    └── asgn1
```

3 Counting Birds With a `calc` Program

Everyone in the bird counting department has, over the years, found the need to implement a simple arithmetic calculator in order to simplify bird-counting operations. At this point, there are 20 bird-counting programs, and there is no need for all of them. In fact, some of them don't work correctly.

Let's take a look.

☐ **To do:** "cd" into the appropriate directory for your environment: "arm," "hub," or "x86."

Type one of these commands:

```
$ cd compiled_cats_hub      ⟵ when on GitHub
$ cd compiled_cats_arm      ⟵ when on a newer Mac
$ cd compiled_cats_x86      ⟵ everything else
```

☐ **To do:** Run the example program `kittycat` below. It prints the sum of its two arguments.

```
$ ls
babycat   coolcat  fluffycat   hungrycat   meowcat   qtcat     sleepycat
blackcat  cozycat  grumpycat   keyboardcat nyancat   scarycat  spacecat
bongocat  fatcat   hairlesscat kittycat    orangecat  sillycat
$ ./kittycat 3 4
7
```

Let's run the program with one non-numeric input.

☐ **To do:** Run the example program `kittycat` with non-numeric input.

```
$ ./kittycat xyz 4
BAD INPUT           ⟵ Yep, xyz isn't a number.
```

Notice that there are twenty different counting programs. Run another one:

☐ **To do:** Run the example program `babycat` below.

```
$ ./babycat 3 4
8                   ⟵ 3 + 4 ≟ 8  Uh, oh. This program has a bug.
```

For this test $(3 + 4)$, the `kittycat` program prints the correct sum, but the `babycat` program does not. Does this mean that the `babycat` program has a bug? Yes it does. Does this mean that the `kittycat` program works correctly? No. We don't know if `kittycat` is *entirely* correct. It may fail *other* tests.

4 The Spec

Among the 20 programs, some correct implementation should be kept (because who wants to write *another* program?). But which program should we keep? The one that works correctly, according to this specification (or "spec"):

A calculator program takes 2 "tiny integers" as command-line arguments, prints their integer sum, and returns a 0 "exit code" (defined in section 5.1) to indicate success. The program checks that its arguments meet the details of this specification. If either argument doesn't, then the program indicates that something went wrong by printing an error and returning an exit code other than 0.

Inputs

- ☐ The program defines a tiny integer as a sequence of one or more digit characters ("0", "1", ..., "9") that has a decimal value between 0 and 512.
- ☐ Optionally, any tiny integer can be preceded by one "+" or "−".
- ☐ If either of the integers is not tiny (that is, the argument is an integer, but its value is less than −512 or more than 512), then the program prints "TOO BIG" and returns a non-zero exit code.
- ☐ A tiny integer cannot start with 0 unless it is "0".
- ☐ Both command-line arguments must be tiny integers.
- ☐ If either command-line argument is not an integer, the program prints "BAD INPUT" and returns a non-zero exit code.
- ☐ If there are fewer than two arguments, the program prints "NOT ENOUGH INPUT" and returns a non-zero exit code.

Outputs

- ☐ The sum is printed as a sequence of digits, with negative sums preceded by a −.
- ☐ The sum may not have any leading space characters.
- ☐ The sum may only start with 0 if it is the value 0.
- ☐ The printed sum is <u>not</u> required to remain inside the range −512 to 512.
- ☐ Every line of output from the program should be followed by a newline.
- ☐ Any time there is an error, the program must return a non-zero "exit code" (see section 5.1).

5 Shell Scripting

You can imagine iteratively running the other eighteen "cats" programs and checking whether they can add 3 and 4 correctly. Or you can write a script.

5.1 Which Commands to Script

Above, while looking at the result of the `kittycat` program, you saw `7`, and you knew that since $3 + 4 = 7$, the program computed that sum correctly. But to automate this procedure, you will need a way to direct the computer to make the comparison itself. The approach is to

1. Store the expected output.
2. Run the program and store its output.
3. Compare the outputs.

☐ **To do:** To carry out these steps, type the commands below.

1. Store the expected output:

```
$ echo 7 > expected.txt   ⟵ Use the echo command to make a file with "7\n".
```

2. Run a program and store its output:

```
$ ./kittycat 3 4 > got.txt   ⟵ Run the program. Store its output in got.txt.
```

3. Compare the outputs. The `echo $?` command prints the "exit code" of the prior command to the screen.

```
$ echo $?                          ⟵ Show the exit code of kittycat (the prior command).
0                                  ⟵ The exit code of the kittycat program. (0 is good.)
$ diff -u expected.txt got.txt     ⟵ Compare the expected output and the program output.
$ echo $?                          ⟵ Let's see the exit code of diff.
0                                  ⟵ The exit code of the diff program. (0 is good.)
```

Okay! After the first `echo $?`, we see that the `kittycat` program returned an exit code of 0. We expected that exit code because we expected the `kittycat` program to report no errors (3 and 4 are valid inputs).

After the second `echo $?`, we see that the `diff` command also returned an exit code of 0. That exit code tells us that the program output (`got.txt`) is the same as the expected output (`expected.txt`).

- The first 0 was expected because we provided valid inputs. (A different test that provided invalid inputs would expect a non-zero value.)
- The second 0 means that the program output is what we expected. (All tests look for a 0 from the `diff` command.)

Remember that each "cat" program has **two** outputs: (1) the output text and (2) the program's "exit code." Any script that you write must confirm that both of these outputs are correct. The variable `$?` represents the prior program's exit code.

☐ **To do:** Now let's test the `babycat` program. The `expected.txt` file already exists, and so instead of recreating it, we'll use it again.

1. Store the expected output (already in `expected.txt`).
2. Run the program and store its output:

```
$ ./babycat 3 4 > got.txt   ⟵ Run the program. Store its output in got.txt.
```

3. Compare the outputs:

```
$ echo $?                          ⟵ Let's see the exit code of babycat.
0                                  ⟵ The exit code of the babycat program. (0 is good.)
$ diff -u expected.txt got.txt     ⟵ Compare the expected output and the program output.
-- expected.txt 2026-01-31 15:15:08.525044213 -0800
+++ got.txt 2026-01-31 15:15:40.645074968 -0800
@@ -1 +1 @@
-7                                  ⟵ The - prefix means that this line is from expected.txt
+8                                  ⟵ The + prefix means that this line is from got.txt
$ echo $?                          ⟵ Let's see the exit code of diff.
1                                  ⟵ The exit code of the diff program. (≠ 0 is bad.)
```

After the first `echo $?`, we see that the `babycat` program returned an exit code of 0 (which is expected because 3 and 4 are valid inputs). However, after the second `echo $?`, we see that the `diff` program reports a difference through a non-zero exit code (1). So because of the second exit code, the `babycat` program didn't pass the test.

5.2 The First Script

We need a way to run a selected "cat" program and report whether it passes the test or not. We've given you a first test script.

☐ **To do:** Go into the `yourcruzid/asgn1/tests` directory.

☐ **To do:** Look at the file `test_3_4.sh`. Three parts of the script are boxed. Here is what they do:

(1) Run the "cat" program with arguments `3` and `4`. We call the program `./calc` because another script copies the tested program to that name. We put the output in `got.txt`. Save the exit code for later.

(2) Generate the expected output using an `echo` command. We put the output in `expected.txt`.

(3) Report when the "cat" program's "exit code" is *wrong*. Does your test give *correct input* like `./calc 3 4` and so expect an exit code of 0? Then the "if" statement should check for the *wrong exit code* ($\neq 0$) using `-ne 0`. Does your test give *incorrect input* like `./calc xyz 4` and so expect an exit code that is $\neq 0$? Then the *wrong* exit code is 0, and the "if" statement should check for $= 0$ using `-eq 0`.

```
./calc 3 4 > got.txt
CAT_EXIT_CODE=$?

echo 7 > expected.txt

if [ $CAT_EXIT_CODE -ne 0 ]; then
    echo "$0: wrong exit code" $?
    rm expected.txt
    rm got.txt

    # Report failure to the calling script and exit.
    exit 1
fi

# Check program output
# Complain if it's unexpected.
diff expected.txt got.txt
if [ $? -ne 0 ]; then
    # message on failure
    echo "$0: wrong output"
    rm expected.txt
    rm got.txt

    # Report failure to the calling and exit.cript.
    exit 1
fi

# We made it this far.
# Celebrate!
echo "$0: PASS"

# Remove temporary files.
rm expected.txt
rm got.txt

# Report success to the calling and exit.cript.
exit 0
```

5.3 Running the First Script

☐ **To do:** Go into the `yourcruzid/asgn1` directory.
☐ **To do:** You run a test script (either a specific one or all of them) with the `runner.sh` script. Run all scripts with the `bash runner.sh` command. Run just `test_3_4.sh` with `bash runner.sh test_3_4.sh`

The all-caps words in the left column show that five of the cats programs failed this first test.

```
$ bash runner.sh test_3_4.sh
PASS: compiled_cats_arm/babycat with tests/test_3_4.sh
FAIL: compiled_cats_arm/blackcat with tests/test_3_4.sh
PASS: compiled_cats_arm/bongocat with tests/test_3_4.sh
PASS: compiled_cats_arm/coolcat with tests/test_3_4.sh
PASS: compiled_cats_arm/cozycat with tests/test_3_4.sh
PASS: compiled_cats_arm/fatcat with tests/test_3_4.sh
PASS: compiled_cats_arm/fluffycat with tests/test_3_4.sh
PASS: compiled_cats_arm/grumpycat with tests/test_3_4.sh
PASS: compiled_cats_arm/hairlesscat with tests/test_3_4.sh
PASS: compiled_cats_arm/hungrycat with tests/test_3_4.sh
FAIL: compiled_cats_arm/keyboardcat with tests/test_3_4.sh
PASS: compiled_cats_arm/kittycat with tests/test_3_4.sh
PASS: compiled_cats_arm/meowcat with tests/test_3_4.sh
PASS: compiled_cats_arm/nyancat with tests/test_3_4.sh
PASS: compiled_cats_arm/orangecat with tests/test_3_4.sh
PASS: compiled_cats_arm/qtcat with tests/test_3_4.sh
PASS: compiled_cats_arm/scarycat with tests/test_3_4.sh
FAIL: compiled_cats_arm/sillycat with tests/test_3_4.sh
FAIL: compiled_cats_arm/sleepycat with tests/test_3_4.sh
FAIL: compiled_cats_arm/spacecat with tests/test_3_4.sh
```

5.4 Create a Second Script

☐ **To do:** Go into the `yourcruzid/asgn1/tests` directory.
☐ **To do:** Copy the first script to make a second script. Call it `test_xyz_4.sh`.

```
$ cp test_3_4.sh test_xyz_4.sh
```

☐ **To do:** The second script currently is identical to the first. Let's edit it. Consult the listing on the prior page with the three numbered parts. In each part, do the following:

(1) In this part, change the first parameter "3" into "xyz".

(2) In this part, change the expected output "7" into "BAD INPUT".

(3) In this part, change "`-ne`" into "`-eq`" so that the script complains when the "cat" program fails to end with a non-zero exit code.

At this point your test file `test_xyz_4.sh` is ready to run.

5.5 How to Run All Scripts

Now you have two scripts. Rather than running one at a time, let's run both of them.
☐ **To do:** Go into the `yourcruzid/asgn1` directory.
☐ **To do:** Run all test scripts with this command: `bash runner.sh`

Look at the output. The all-caps words in the left column show that one more of the cats programs failed this second test.

☐ **To do:** The `runner.sh` script has three internal parameters. You can edit `runner.sh` to control these parameters. Set `PRINT_TEST_OUTPUT=1` to see the `diff` outputs.

```
# Configure the test script.
REPORT_PASSING_TESTS=1
PRINT_TEST_OUTPUT=0
PRINT_USING_COLOR=1
```

6 Your Task

Your task in this assignment is to help Brad find which of the implementations of the bird counting programs follow the specifications. To do this, you will be writing tests, and using `runner.sh` to run them on all of the provided programs.

6.1 Write Test Scripts

☐ **To do:** Write a collection of shell scripts that test the various implementations of the calculator program. You already have `test_3_4.sh` and `test_xyz_4.sh` in the `tests` directory. You may write as many scripts as you like, and may give them whatever names you like (though we recommend intuitive names that say something about what the script checks). Each script must work in the same way as `test_3_4.sh`:

- Each script must be stored in the `tests` directory.
- Each script must be named `test`*something*`.sh`
- Each script can be run at the command line with no arguments.
- Each script may assume that the calc program has been compiled, is in the current directory, and is named "calc".
- The script should test calc in some way by running it and checking its output.
- Each script should communicate a successful test by returning an exit code of 0, and a failed test by returning an exit code other than zero. When a test fails, it is helpful to print something that describes the error.
- Each script should not have false positives, meaning it should not report a bug on a correctly working program.

6.2 Scoring

☐ **To do:** Create more test scripts in the `tests` directory. (if your tests are simple, you probably will need many to get the maximum score.)

We will run your scripts against a collection of working and buggy programs. Your score will be based on how many of the buggy programs your scripts identify. Remember that in order for a script to identify a bug, it must return a non-zero number if the provided `calc` program does not follow the spec. Only one script needs to identify the buggy program to receive credit for it.

It is important that your scripts do **not** report false positives (i.e. they should exit with 0 on all the working programs). Any script that reports a working program as buggy will *not* be considered for grading. If you do have false positives, it is recommended to fix them first.

Why is this rule necessary? Consider a very simple script that just contains `exit 1`. This script reports a bug on *every* program. Such a script would identify all the buggy programs, but it obviously is not a proper test script.

6.3 questions.txt

We have given you a file called questions.txt. Edit the file and add your answers to these two questions. **In the questions, when we ask about the program `calc`, we are referring to the twenty "cat" calculator programs that you tested.**

6.3.1 Question 1

> *Describe a "test plan" for the various implementations of calc. Given that a program cannot be tested on every possible input, what are three examples of tests that are implied by the spec but not checked by `test_3_4.sh`? (If you already have written your tests, you just need to describe what three of them are checking for.)*

☐ **To do:** Include your answer to Question 1 in `questions.txt`.

6.3.2 Question 2

> *You may have noticed that the spec says nothing about what to do if the user supplies too many arguments at the command line. (Unfortunately, ambiguity in specifications is very common in practice and can be very confusing for programmers.)*
>
> *If you were asked to implement `calc` given this spec, you would be forced to choose from among at least three interpretations of what `calc` would do when called as, say, `./calc 3 4 5`*
>
> - `calc` *should sum all of the arguments (and print 12).*
> - `calc` *should ignore the third argument (and print 7).*
> - `calc` *should print an error message and return non-zero.*
>
> *Below, state whether your test scripts check what `calc` does when given more than two arguments. Explain why you wrote your scripts that way.*

☐ **To do:** Include your answer to Question 2 in `questions.txt`.

6.4 Submit: add/commit/push/submit commit ID

You need to submit these files:

- `runner.sh`
- `tests/test_*.sh`
- `questions.txt`

Please do not submit either of the `compiled_cats` folders.

☐ **To do:** `git add` all of the files above.
☐ **To do:** `git commit -m '`*reason*`'` (you can give any reason for this commit)
☐ **To do:** `git push`
☐ **To do:** Run the git pipeline to check your test results.
☐ **To do:** Submit your final commit ID to Canvas by Wednesday, October 14th at 11:30 pm.

7 Revisions

Version 1 Original.